MC 451 Data Wrangling in R rubric

50 points. Due Friday, November 13, 11:59 PM.

How this is scored. Add the points from each row; the total is out of 50. A first attempt is graded forgivingly: a part that is there but incomplete earns partial credit, and 0 is kept for a part that is missing. Length: one script of 40 to 120 lines, comments included, plus the 30-row coding form. Word counts cover your own writing, not the title, headings, tables, references, figure captions, code, or output, and writing more than the range never costs points. Scores on work that shows effort are adjusted with the course's curve: your final percentage is 25 percent plus 75 percent of your rubric percentage, so 50 of 100 on the rubric becomes 62.5. Work at two-thirds or more of the minimum length counts as showing effort. Submissions are unlimited before the deadline, so you can revise and submit again.

- Inferencing Data needs all four kinds of variable from this table: two categorical, one with three or more groups, and two continuous. Step 5 is where they are kept.
- What it feeds: Describing Data and Inferencing Data both read `twitch_analysis.RDS`. Your White Paper reuses the moves in step 1 (read a coding form in, make the codes factors, join by id) on your own data; the Twitch table itself stays here.
- Code the AURA Lab R Companion writes into your script counts as script code. Keep the comment line it writes above each block, and be ready to explain every line.
- Before you submit: restart R, run the script top to bottom, and see what breaks.

Data Wrangling in R, 50 points

Criterion	Exemplary	Proficient	Developing	Beginning
1. Hand-code, and read your codes back 6 points	**6 points.** One variable from your codebook hand-coded for a 30-message sample drawn with `sample_messages()`, recorded in a coding-form spreadsheet (one row per message, one column per variable), read back with `read_csv()`, its column names tidied with `clean_names()`, the codes turned into a labeled factor with `factor()`, and joined to the sample by message id.	**5 points.** Hand-coded and read back, but the codes stay as bare numbers or text with no `factor()`, or the sample was not drawn with `sample_messages()` or is not 30 messages.	**3 to 4 points.** Coded with a variable that is not in your codebook, or the codes are typed into the script instead of read from a coding form.	**0 to 2 points.** No hand-coding.
2. Load 6 points	**6 points.** The chat and stream tables are loaded from the v2v package inside the script.	**5 points.** Both loaded, but from a file on your own computer, so the script fails anywhere else.	**3 to 4 points.** Only one table loaded.	**0 to 2 points.** Not loaded in the script.
3. Convert the timestamps 6 points	**6 points.** Epoch milliseconds converted to a real date-time with `clean_dates()`, which divides by 1,000, or by hand, dividing by 1,000 first.	**5 points.** Converted, but a small error remains, such as times off by some hours because the time zone was not set.	**3 to 4 points.** Converted without dividing by 1,000, so the dates land thousands of years away.	**0 to 2 points.** Not converted.
4. Message length 6 points	**6 points.** A new column holding each message's length.	**5 points.** Length computed, but on the wrong field (for example, the username).	**3 to 4 points.** Length computed but not saved as a column.	**0 to 2 points.** Missing.

Criterion	Exemplary	Proficient	Developing	Beginning
5. Join, and keep the analysis columns 6 points	**6 points.** Stream information joined onto the chat messages, with a comment naming the key you joined on, and the table keeps the channel's main category (main_game), `is_gaming`, `viewer_band`, and one continuous audience measure (`median_viewers`, or each message's viewers from the snapshot nearest before it, which `join_chat_streams()` attaches).	**5 points.** Joined and the columns kept, but no comment names the key, or the continuous audience measure is missing.	**3 to 4 points.** The join runs but matches nothing or multiplies the rows, and the script does not notice; or only `is_gaming` survives, so the table supports a t-test and nothing else.	**0 to 2 points.** No join.
6. Missing values 6 points	**6 points.** Reports how many rows carry missing values and what you did with them.	**5 points.** Handled, but the count is not reported.	**3 to 4 points.** Rows with missing values dropped without saying how many. Dropping them silently is the error this step exists to catch.	**0 to 2 points.** Not handled.
7. Export 6 points	**6 points.** Saved as `twitch_analysis.RDS`.	**5 points.** Saved under another name, so the next two assignments cannot find it.	**3 to 4 points.** Saved as a CSV or another format that loses the column types.	**0 to 2 points.** Not exported.
8. Every step is in the script 6 points	**6 points.** Every change to the data happens in the script; nothing is done by hand in the console.	**5 points.** One step was done outside the script, but a comment says what and how.	**3 to 4 points.** Several steps were done outside the script.	**0 to 2 points.** The script does not rebuild the dataset.

Criterion	Exemplary	Proficient	Developing	Beginning
Clean-session run 2 points	**2 points.** Runs start to finish after restarting R.	**1 point.** Runs after one small fix, such as a missing `library()` line.	**0 points.** Stops partway in a fresh session.	**0 points.** Runs only with objects already in your environment, or not at all.